

ABRA-CAPA-DABRA: Full break of CAPA

Paco Azevedo-Oliveira^{1,2}

¹ Thales, France

paco.azevedo-oliveira@thalesgroup.com

² Laboratoire de Mathématiques de Versailles, UVSQ, CNRS, Université Paris-Saclay, 78035 Versailles, France

Abstract. In a paper presented at CRYPTO18, Reparaz *et al.* introduce a new model called the “Tile-Probe-and-Fault-Model”. This model proposes to cover more realistic side-channel attacks by considering so-called “combined” attacks, in other words attacks using both side-channel leakage and fault injection. In addition, they introduce CAPA, a combined “Countermeasure Against Physical Attacks”, which is inspired by an MPC protocol called SPDZ. In this paper, by attacking the original Beaver triples generation described in CAPA, we demonstrate that the CAPA countermeasure is not secure in its model, by breaking the countermeasures with a simple resolution of a linear system.

Keywords: Probing Security, Side-Channel Analysis, Cryptanalysis.

1 Introduction

For more than twenty years, side-channel attacks [Koc96] have represented one of the most powerful threats to embedded products that use cryptographic protocols. Although an algorithm is considered secure in the black-box model, its security can collapse as soon as an attacker has access to auxiliary information (via electrical consumption, an electromagnetic probe, etc). Numerous techniques have been developed to counter these powerful attacks, the main one being masking [DFS15].

However, if an attacker can observe through hidden channels, he can often disrupt the algorithm by injecting faults during its execution [BDL97]. As these attacks can also be dangerous, there are many countermeasures to secure implementations against an active attacker, the most commonly used being the introduction of redundancy and the use of error-correcting code.

Unfortunately, theoretical models and countermeasures usually consider separate attackers. More specifically, there are countermeasures for covert channel attacks or (exclusive) faults, yet few papers study the fact that a countermeasure against faults (*e.g.* adding redundancy) can facilitate auxiliary channel attacks, and vice versa.

It was to remedy this that [RDB⁺18] was written. It is one of the first papers to define a model (known as the “Tile-Probe-and-Fault-Model”) in which the attacker is able to use auxiliary and fault analysis tools simultaneously. In addition to this new model, they proposed a generic countermeasure called CAPA, for “Combined countermeasure Against Physical Attacks”, which makes it possible to secure a program (in hardware or software) in the “Tile-Probe-and-Fault-Model”. As such a model is naturally close to multiparty computation (MPC), CAPA is in fact an adaptation of an MPC protocol called SPDZ [DPSZ12], but which takes advantage of a restricted attacker model (*i.e.* less strong than that of the active adversary in MPC) to be more efficient.

More precisely, SPDZ is an MPC scheme that uses secret sharing to compute a function whose inputs are private. The protocol is split into two parts, the “offline” phase, during which the players generate correlated randomness in the form of triples (a, b, ab) that are shared among them; and the online phase during which the triples generated previously are used as disposable masks to apply the function without revealing anything about its inputs. Generating triples in SPDZ is the most expensive part of the process, as it requires the use of homomorphic encryption schemes and zero-knowledge proofs. CAPA is identical to SPDZ except for the offline phase, which is much less costly thanks to a weaker attacker model.

Related work. An article, [TNN24] has already proposed an attack against CAPA in its security model. This paper proposes a combined attack (fault-based + probing) against CAPA that have a probability 2^{-k} to work, where k is the size of the finite field and suggests countermeasures to mitigate this attack. Our result generalizes this, showing that there is no need to inject any faults to break the protocol. We show that triplet generation is by construction insecure: it reveals the triplet that will be used, and it is enough to observe the protocol without having to disturb it. In other words, the protocol is not even secure in a passive attacker’s model.

Our Contributions. This paper shows that CAPA is not secure in the “Tile-Probe-and-Fault-Model”. More precisely, we show that the generation of CAPA triples is not secure: it is possible to find the triples at time of their generation, which implies that the security of the online phase collapses. In addition, we show that it is not so easy to repair the generation of triples in CAPA, by showing that some of the natural ideas for reparation are impossible.

Outline. In Section 2, we review some definitions of multiparty computation and recall the basic knowledge of the SPDZ scheme, as well as the CAPA countermeasure. Section 3 proposes various attacks on CAPA. Finally, Section 4 contains a short conclusion.

2 Background and notations

2.1 The Tile-Probe-and-Fault-Model

This sub-section provides only a summary of the “Tile-Probe-and-Fault-Model”, for a full description the reader is referred to [RDB⁺18].

2.1.1 Tiles, Probes and Faults

Throughout the rest of the paper, the code or chip that implements a cryptographic function is considered to be partitioned into d tiles $\mathcal{T}_1, \dots, \mathcal{T}_d$. Each tile $\mathcal{T}_i$ is considered independent of the others: it has its own code, its own random number generator, its own memory and its own logic functions for performing calculations. The different tiles can interact with one another via wires and communicate in order to carry out a common calculation, typically applying a cryptographic function. The model described above is shown in Figure 1.

In addition, the fact of “probing” a tile is defined as the recovery, with probability 1, of all the intermediate values of the tile, for the entire duration of the calculation. There is a distinction between two types of fault: “selected-value faults” and “random-value faults”. According to [RDB⁺18], selected-value faults “have the nature of either flipping the intermediate values with a pre-calculated (adversarially chosen) offset or setting the intermediate values to a chosen fixed value. In particular, the faults are not limited in

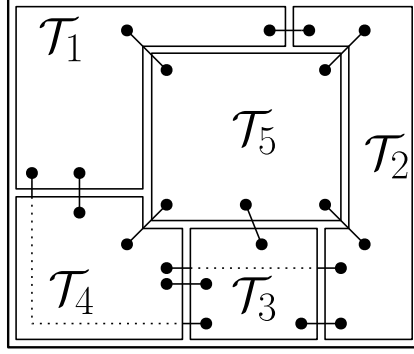


Figure 1: Diagram of the Tile-Probe-and-Fault-Model

hamming weight. One can relate this type of faults with, for example, very accurate laser injections.”. While random-value faults are defined as “injecting the random-value fault according to some distribution (for example, flip each bit with certain probability), but he cannot set all intermediates to a chosen fixed value.”. These two types of fault give rise to two types of attacker, formally defined below.

2.1.2 Attacker model

The first attacker, $\mathcal{A}_1$, is capable of probing $d_p \in \{1, \dots, d-1\}$ tiles and making chosen-value faults on $d_f \in \{1, \dots, d-1\}$ tiles. Let $\mathcal{P}_1$ be the set of tiles which can be probed and $\mathcal{F}_1$ the set of tiles where $\mathcal{A}_1$ can inject faults. $\mathcal{A}_1$ ’s attack surface is restricted by the condition:

$$|\mathcal{F}_1 \cup \mathcal{P}_1| \leq d-1,$$

in other words: at least one tile remains neither probed nor attacked.

The second, $\mathcal{A}_2$, is an attacker capable of probing $d_p \in \{1, \dots, d-1\}$ tiles and injecting random-value faults into all the tiles. If $\mathcal{P}_2$ denotes the set of tiles that $\mathcal{A}_2$ can probe, the condition $|\mathcal{P}_2| \leq d-1$ applies, but there is no restriction on the size of $\mathcal{F}_2$. It is important to note that the attacker model described here is more powerful than the standard “t-probing” and “ ϵ -probing” models. In this model, the attacker has access to a significantly larger number of intermediate variables. However, this model is not as strong as the MPC model, since the attacker can never completely deviate from the protocol. More precisely, according to [RDB⁺18]: “A rough equivalent model in the MPC literature would be for an honest-but-curious adversary who is able to replace the share or MAC values of honest players with values selected from a given random distribution. Whilst such an attack makes sense in the hardware model we consider, in the traditional MPC literature this model is of no interest due to the supposed isolated nature of computing parties.”.

2.2 A countermeasure using CAPA

CAPA is a countermeasure against a tile-probe-and-fault-attacker, described in [RDB⁺18] which is built upon SPDZ [DPSZ12]. SPDZ is a protocol that allows a set of d parties to evaluate a public function f without revealing anything other than the output of this function, even if a malicious adversary controls up to $d-1$ participants. The protocol can be used to calculate any circuit in any finite field $\mathbb{F}$ of characteristic 2, and consists of two phases, an “offline” phase and an “online” phase. Formally, assume $\mathcal{T}_1, \dots, \mathcal{T}_d$, d participants such that each $\mathcal{T}_i$ knows $x_i \in \mathbb{F}$, for some finite field $\mathbb{F}$. The objective for the

$\mathcal{T}_i$'s is to jointly calculate $f(x_1, \dots, x_d)$, where $f : \mathbb{F}^d \rightarrow \mathbb{F}$. One can simplify the problem: It's enough to calculate $(x, y) \rightarrow x + y$ and $(x, y) \rightarrow xy$ safely. In what follows, SPDZ is not described, we focus only on CAPA. The following can be read without further reference, but is described in more detail in [RDB⁺18] and in [DPSZ12].

Splitting secrets: In the rest of the paper, we focus on $\mathbb{F} := \mathbb{F}_{2^k}$ a finite field of characteristic 2 and of size 2^k . A secret value $x \in \mathbb{F}$ is shared between the player (or tiles) $\mathcal{T}_1, \dots, \mathcal{T}_d$ using:

$$\langle x \rangle = (\mathbf{x}, \boldsymbol{\tau}^x)$$

where $\mathbf{x} = (x_1, \dots, x_d)$ is called the data shares while $\boldsymbol{\tau}^x = (\tau_1^x, \dots, \tau_d^x)$ is called the tag shares. For $i \in \{1, \dots, d\}$, $\tau_i^x \in \mathbb{F}^m$, where $m \in \mathbb{N}^*$ is called the security parameter of CAPA. Each $\mathcal{T}_i$ knows the triple $(x_i, \tau_i^x, \boldsymbol{\alpha}_i)$, which verify for all $j \in \{1, \dots, m\}$:

$$x = \sum_{i=1}^d x_i \quad \text{and} \quad (x \cdot \boldsymbol{\alpha})[j] = \sum_{i=1}^d \tau_i^x[j],$$

where $\boldsymbol{\alpha} \in \mathbb{F}^m$ is called a MAC key and is also shared among the player with $\boldsymbol{\alpha} = \sum \boldsymbol{\alpha}_i$.

Reveal a secret value: Let $x \in \mathbb{F}_q$ be a secret value shared among the $\mathcal{T}_i$'s under the form $\langle x \rangle$. To reveal x , each $\mathcal{T}_i$ broadcasts the value x_i , so each $\mathcal{T}_i$ can calculate $x = \sum x_i$. $\boldsymbol{\tau}^x$ is used for verification: each $\mathcal{T}_i$ calculates and then broadcasts $x \cdot \boldsymbol{\alpha}_i + \tau_i^x$. If the tag for x is correct, one expects:

$$\sum_{i=1}^n x \cdot \boldsymbol{\alpha}_i + \tau_i^x = 0.$$

If it is not, one of the $\mathcal{T}_i$'s sends a message which interrupts the rest of the calculation.

Calculate the sum of secret values: Let $x, y \in \mathbb{F}$ be two secret values shared between the $\mathcal{T}_i$'s and represented by their respective forms $\langle x \rangle$ and $\langle y \rangle$. The addition of $\langle x \rangle$ and $\langle y \rangle$ can be done locally (*i.e.* without communication between the $\mathcal{T}_i$'s) by calculating:

$$(x + y)_i := x_i + y_i \quad \text{and} \quad \tau_i^{x+y} := \tau_i^x + \tau_i^y.$$

This produces a shared form of $x + y$, denoted $\langle x + y \rangle$.

Calculate the product of secret values: Let $x, y \in \mathbb{F}$, represented as $\langle x \rangle$ and $\langle y \rangle$. To calculate $\langle xy \rangle$, the $\mathcal{T}_i$'s must first know a “Beaver triple”, which is a triple of the form $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ where a, b are random and $c = ab$. The $\mathcal{T}_i$'s begin by revealing $\langle x - a \rangle$ and $\langle y - b \rangle$, and a split of the product can be calculated using:

$$\begin{cases} (xy)_i = c_i + (x - a)b_i + (y - b)a_i + (x - a)(y - b)\delta_i^1 \\ \tau_i^{xy} = \tau_i^c + (x - a) \cdot \tau_i^b + (y - b) \cdot \tau_i^a + (x - a)(y - b) \cdot \boldsymbol{\alpha}_i \end{cases}$$

where δ_i^1 is 0 except when $i = 1$.

Remark 1. *This shows that calculating a product of selected shared values is just as difficult as calculating the product of random ones, furthermore Beaver triples are independent of the calculations performed during the online phase and can therefore be generated before or during the calculations.*

2.3 CAPA's custom Beaver triple generation

The main difference between CAPA and SPDZ is in the way triples are generated: SPDZ uses a SHE (somewhat homomorphic encryption) scheme, where each $\mathcal{T}_i$'s need it's own public key. CAPA, on the other hand, offers an original, less heavy method of generating triples: the $\mathcal{T}_i$'s generate the triples without checking their validity (with some passively secure shared multiplier), then they use a “sacrifice” principle to verify the validity of a triple. Let $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ and $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ be two potentially incorrect triple, *i.e.* such that $c = ab + \Delta_c$ and $f = ed + \Delta_f$, CAPA describes a method for proving that the triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is functional (*i.e.* $\Delta_c = 0$) by revealing the triple $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$. This method is formally described below:

1. Draw a random $r^{(1)} \in \mathbb{F}$.
2. Each $\mathcal{T}_i$ computes $\langle r^{(1)}a \rangle = r^{(1)} \cdot \langle a \rangle$.
3. The $\mathcal{T}_i$'s use $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ to compute $\langle \tilde{c} \rangle := \langle (r^{(1)}a)b \rangle$.
4. Each $\mathcal{T}_i$ computes the difference with the shares and tags of $r^{(1)}c$ and $\tilde{c}$ by computing:

$$\Delta_i := r^{(1)}c_i + \tilde{c}_i \quad \text{and} \quad \tau_i^\Delta := r^{(1)} \cdot \tau_i^c + \tau_i^{\tilde{c}}.$$

5. The $\mathcal{T}_i$'s reveal Δ and τ^Δ .
6. If $\Delta \neq 0$ or $\tau^\Delta \neq 0$, reject $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$.
7. Restart with $r^{(2)} \in \mathbb{F}$ such that $r^{(1)} \neq r^{(2)}$. If $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ has not been rejected, it is considered valid.

Remark 2. *In the words of [RDB⁺18]: “Note that this relation verification ensures that the second triple is functionally correct too. However, it is burnt (or “sacrificed”) in this process in order to ensure that the first triple can be used securely further on.”. Furthermore, [RDB⁺18] proves that the sacrifice protocol is functional. More precisely, their results can be reformulated with the Assertion 1:*

Assertion 1. *[RDB⁺18] Let $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ and $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ be two potentially incorrect triple, the principle of sacrifice ensures that triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is functional with probability $1 - 2^{-km}$ at the cost of revealing $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$.*

3 Attack description

The aim of this section is to propose a rigorous proof of Assertion 1. By formalising the proof, we realise that the original sacrifice principle written in CAPA is not applicable: it reveals too much information about triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$, even when the protocol is executed correctly.

Theorem 1. *Assertion 1 is incorrect. More precisely, let $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ and $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ be two potentially incorrect triple. The principle of sacrifice ensures that triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is functional with probability $1 - 2^{-km}$ at the cost of revealing $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ **and** $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$.*

Proof. Let $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ and $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ be two triples, assumed to be functional (*i.e.* $c = ab$ and $f = ed$) and that the sacrifice protocol is executed correctly. For $r^{(1)} \in \mathbb{F}$, to calculate $\langle r^{(1)}a \rangle \cdot \langle b \rangle$ with the triple $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ the $\mathcal{T}_i$'s must reveal $r^{(1)}a - d$ and $b - e$. When the triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ passes the sacrifice stage, the $\mathcal{T}_i$'s have revealed:

$$\epsilon^{(1)} = r^{(1)}a - d, \quad \epsilon^{(2)} = r^{(2)}a - d, \quad \eta^{(1)} = \eta^{(2)} = b - e.$$

Therefore, a is known from all the $\mathcal{T}_i$'s:

$$a = (\epsilon^{(1)} + \epsilon^{(2)})(r^{(1)} + r^{(2)})^{-1}. \quad (1)$$

At the end of the sacrifice protocol, the $\mathcal{T}_i$'s knows a , d and $b - e$. Only by observing valid triples an adversary $\mathcal{A}$ knows the value of a , the triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ can no longer be used in the online phase, as this would reveal part of the secret data. $\square$

Remark 3. *The attacker defined in Theorem 1 is significantly less powerful than the Tile-Probe-and-Fault-Model allows. It is sufficient that it has access to the data of only one of the tiles $\mathcal{T}_i$ and worse still is not supposed to be active either before, during or after the protocol.*

Revealing secrets: During the online phase, if the implemented function is non linear, there will be at least one multiplication performed on sensitive values, denoted x and y and shared among the tiles as $\langle x \rangle$ and $\langle y \rangle$. To perform this multiplication, the tiles follow the protocol described in Section 2 by choosing two triples $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ and $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ generated during the offline phase, then using the sacrifice principle to ensure that $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is correct. They then begin by revealing $x - a$ and $y - b$ in order to obtain a share of the multiplication xy . According to Theorem 1, any player can find a and can then retrieve the secret value x .

Remark 4. *The attack succeeds when the attacker has access to all intermediate values of a tile from the beginning to the end of the calculations. However, it is important to note that the attacker may restrict himself to certain time intervals. For the attack to succeed, the attacker must at least know the intermediate values of a tile during step 1, step 3, and sub-steps 1 and 3 of step 7 in the sacrifice principle.*

3.0.1 Infeasible reparations

In what follows, we show that it is not possible to secure the sacrifice protocol using the most intuitive method, that which consists of sacrificing more than one triple.

Proposition 1. *Let $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$, $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ and $(\langle g \rangle, \langle h \rangle, \langle i \rangle)$ be three potentially incorrect triples, the principle of sacrificing which consists in sacrificing $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ in the first round and $(\langle g \rangle, \langle h \rangle, \langle i \rangle)$ in the second round does not provide sufficient security.*

Proof. Let $\mathcal{A}$ be an attacker who controls one of the $\mathcal{T}_i$'s. Without loss of generality we assume that $\mathcal{A}$ controls $\mathcal{T}_1$. When generating triples (which is done using a passively secure shared multiplier in CAPA), $\mathcal{T}_1$ can deviate from the protocol to obtain three triples $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$, $(\langle d \rangle, \langle e \rangle, \langle f \rangle)$ and $(\langle g \rangle, \langle h \rangle, \langle i \rangle)$ such that:

$$c = ab + \Delta_c, \quad f = ed + \Delta_f, \quad \text{and} \quad i = gh + \Delta_i.$$

Furthermore, if $\mathcal{T}_1$ deviates from the protocol only when calculating c , f and i , the macs τ^c, τ^f, τ^i are valid macs for the incorrect values c , f and i . Therefore, the triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ will be considered correct by the double sacrifice protocol if:

$$\begin{cases} \Delta^{(1)} = \sum_{j=1}^d \Delta_j^{(1)} = \sum_{j=1}^d r^{(1)} c_j + \tilde{c}_j = r^{(1)} \Delta_c + \Delta_f = 0 \\ \Delta^{(2)} = \sum_{j=1}^d \Delta_j^{(2)} = \sum_{j=1}^d r^{(2)} c_j + \tilde{c}_j = r^{(2)} \Delta_c + \Delta_i = 0 \end{cases}$$

and

$$\begin{cases} \tau^{\Delta^{(1)}} = \sum_{j=1}^d \tau_j^{\Delta^{(1)}} = \sum_{j=1}^d r^{(1)} c_j + \tilde{c}_j = (r^{(1)} \Delta_c) \cdot \alpha + \Delta_f \cdot \alpha = 0 \\ \tau^{\Delta^{(2)}} = \sum_{j=1}^d \tau_j^{\Delta^{(2)}} = \sum_{j=1}^d r^{(2)} \tau_j^c + \tau_j^{\tilde{c}} = (r^{(2)} \Delta_c) \cdot \alpha + \Delta_i \cdot \alpha = 0 \end{cases}$$

where $\Delta^{(j)}$ and $\tau^{\Delta^{(j)}}$ denote the Δ and τ^{Δ} of the j -th round.

Even if the triple $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is incorrect, it will be accepted as valid as soon as $\Delta_f = r^{(1)} \Delta_c$ and $\Delta_i = r^{(2)} \Delta_c$. Despite repetition, $\mathcal{A}$ can force acceptance of incorrect triples by fixing $\Delta_f = r^{(1)} \Delta_c$ and $\Delta_i = r^{(2)} \Delta_c$. Finally, even if $r^{(1)}$ and $r^{(2)}$ are assumed unknown to $\mathcal{A}$, and that he sets the values of Δ_f and Δ_i at random, he has probability 2^{-2k} of success. $\square$

Remark 5. *The modified version of the sacrifice protocol does not allow the selection of correct triples. Firstly, the proof assumes a much less powerful attacker than the Tile-Probe-and-Fault-Model allows. An attacker only has to corrupt one of the tiles and does not even need to disrupt the execution of the sacrifice principle. Despite a less powerful attacker, the sacrifice protocol has a probability 2^{-2k} of selecting triples that are incorrect. This probability no longer depends on m the security parameter in CAPA but only on the size of the field $\mathbb{F}$. This probability is too high for the sacrifice principle to be used safely, particularly in small fields.*

4 Conclusion and discussion

In this paper we have shown the weakness of CAPA, a countermeasure presented in [RDB⁺18] at CRYPTO18. These attacks do not call into question [RDB⁺18]’s main contribution, *i.e.* the “Tile-Probe-and-Fault-Model”, but show that the pre-processing stage for CAPA needs to be redesigned. The attack described is constructed in this particular model, it is therefore a theoretical attack. Carrying out the attack in practice would be a more difficult task that has not been attempted here. Nevertheless, it seems important to raise a few points that would make the transition from theory to practice difficult.

- The linear equation (1) never appears explicitly in the calculations; it is necessary to find the values $r^{(1)}$, $r^{(2)}$, $\epsilon^{(1)}$, and $\epsilon^{(2)}$ in order to find a .
- The values $r^{(1)}$ and $r^{(2)}$ are considered known to the attacker in [RDB⁺18]. They are not protected, and the $\mathcal{T}_i$ must generate and exchange these values, which means there is a potential for information leakage. However, $r^{(1)}$ and $r^{(2)}$ are random and only used in step 2 of the sacrifice protocol. Furthermore, no additional information can be obtained on the joint generation of randomness because the paper does not specify a way to generate these random values.
- Similarly, for $\epsilon^{(1)}$ and $\epsilon^{(2)}$, these values are public in the model, but are only revealed and exchanged by the parties at step 3 of the sacrifice protocol.

It seems that a passive attack recovering these four values will necessarily be a high-order attack that combines several points of interest in time. Nevertheless, a side-channel attack is always performed on a specific code and not on a theoretical protocol. To draw definitive conclusions, it would be necessary to have the code for an implementation that uses the CAPA countermeasure.

References

- [BDL97] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of checking cryptographic protocols for faults (extended abstract). In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 37–51. Springer, Heidelberg, May 1997.
- [DFS15] Alexandre Duc, Sebastian Faust, and François-Xavier Standaert. Making masking security proofs concrete - or how to evaluate the security of any leaking device. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 401–429. Springer, Heidelberg, April 2015.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Heidelberg, August 2012.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In Neal Koblitz, editor, *CRYPTO'96*, volume 1109 of *LNCS*, pages 104–113. Springer, Heidelberg, August 1996.
- [RDB⁺18] Oscar Reparaz, Lauren De Meyer, Begül Bilgin, Victor Arribas, Svetla Nikova, Ventzislav Nikov, and Nigel P. Smart. CAPA: The spirit of beaver against physical attacks. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 121–151. Springer, Heidelberg, August 2018.
- [TNN24] Dilara Toprakhisar, Svetla Nikova, and Ventzislav Nikov. Capabara: A combined attack on capa. In Romain Wacquez and Naofumi Homma, editors, *Constructive Side-Channel Analysis and Secure Design*, pages 76–89, Cham, 2024. Springer Nature Switzerland.